

Lecture02: Ambiente de desarrollo y Adquisición de datos

Yomin E. Jaramillo Múnera
EAFIT



Contenido

1. Ambiente de desarrollo
2. Knowledge refresh (numpy & pandas)
3. Adquisición y limpieza de datos



Ambiente de desarrollo

Este curso se desarrollará en **Python**

- Amplio Soporte y Ecosistema
- Sintaxis Legible y Concisa
- Integración con otros Lenguajes



Ecosistema de Librerías Especializadas en ML:

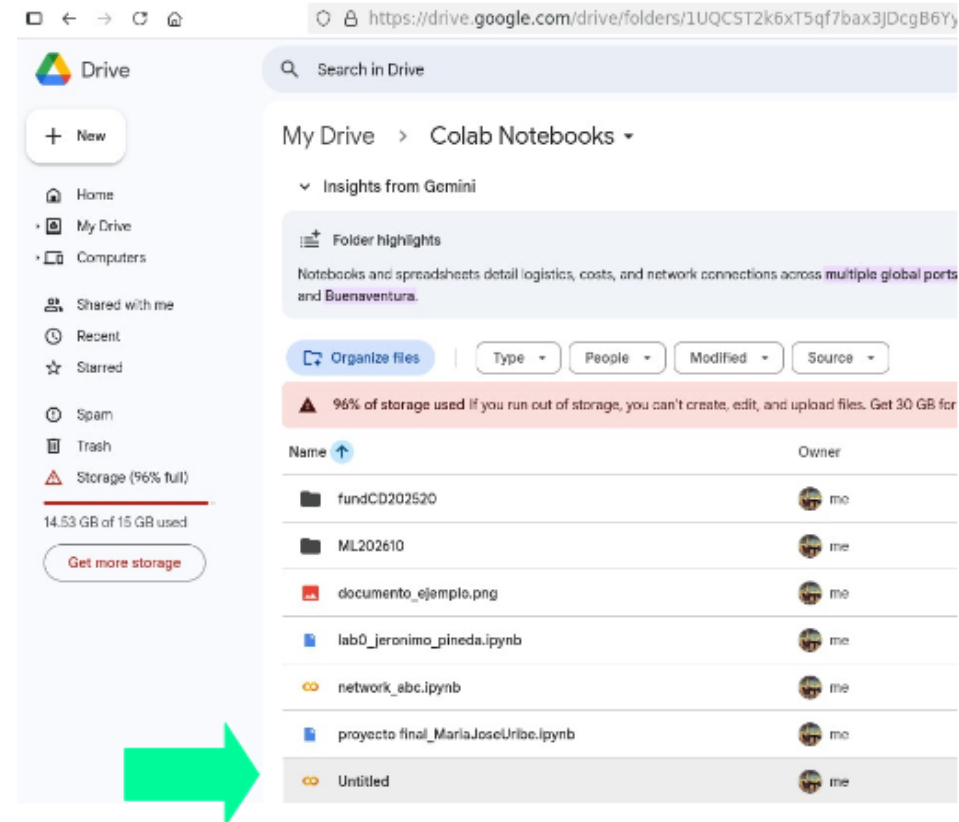
- **Pandas y NumPy:** Fundamentales para la manipulación y análisis de datos estructurados.
- **Matplotlib y Seaborn:** Para la visualización de datos, esencial para entender los modelos.
- **Scikit-learn:** Para algoritmos clásicos de ML (regresiones, clasificaciones, clustering).



Ambiente de desarrollo

Posibles IDEs:

Google Colab
Lightning.ai
VS code
Pycharm



Trabajar en notebooks (extensión .ipynb)



Ambiente de desarrollo

Ventajas de plataformas Online:

Cambiar tipo de entorno de ejecución

Tipo de entorno de ejecución

Python 3

Acelerador por hardware ?

☐ CPU ☒ GPU T4 ☐ GPU H100 ☐ GPU A100

☐ GPU L4 ☐ TPU v5e-1 ☐ TPU v6e-1

¿Quieres acceder a GPUs premium?
[Compra unidades de computación adicionales](#)

Versión del entorno de ejecución ?

Última (recomendada)

Cancelar Guardar

Choose GPU machine

Running on

Interruptible ☐

Save 50-80% with interruptible (spot) studios. [Learn more](#)

Quantity	Model	Speed (TFLOPs)	Memory (GB)	CPU	Cost (hour)	Wait time (min)
1 4	T4	65	16	4	0.43	4
1 2 4 8	L4	121	24	8	0.48	3
1 4 8	L40S	362	48	16	2.14	4
1 2 4 8	RTXP 6000	500	96	16	5.39	3
1 2 4 8	A100	312	40	30	5.68	3
1 2 4 8	H100	1979	80	24	12.34	307
1 8	H200	1979	141	16	6.53	13
1 8	B200	0.00	0	20	9.86	8

You are currently using this machine which has 1 GPU

Request



Ambiente de desarrollo

Instalación de librerías

En Colab

```
!pip install nombreLibrería
```

Local

```
pip install nombreLibrería
```



Ambiente de desarrollo

Respecto al manejo de archivos en Colab

Si deseamos montar nuestro Drive, ejecutamos:

```
#Montamos el drive de google.  
from google.colab import drive  
drive.mount('/content/drive')
```



Nota: Al ejecutar lo anterior, debemos darle permiso a Google para acceder a nuestro Drive desde Colab



Actividad

- 1) Acceder a Google Colab.
- 2) Ejecutar una celda y verificar el panel de Variables.
- 3) Ejecutar varias celdas y verificar el panel de Variables.
- 4) Escoger GPU como entorno de ejecución.
- 5) Montar el Drive personal en Colab
- 6) Cargar un excel en Colab usando Pandas.
- 7) Consultar cómo sabemos si tenemos una librería instalada ya sea en Colab en un ambiente de desarrollo local.



Knowledge refresh

Numpy

Para que sirve?

NumPy (Numerical Python) es la librería fundamental de Python para la computación científica y matemática. Su función principal es proporcionar una estructura de datos muy eficiente llamada *array* (arreglo) para manejar grandes volúmenes de datos numéricos, vectores y matrices



Knowledge refresh

Numpy

```
✓ import numpy as np
  import matplotlib.pyplot as plt

lista=[8,9,10,20,50]
vector=np.array([20, 50, 82, 95])
#print(vector.shape)

matriz=np.array([[1,2,3],[5,6,7],[8,9,10]])
#print(matriz)
#print(matriz.shape)

✓ tensor= np.array([
    [[0,0,0], [0,0,0], [0,0,0]],
    [[128, 128, 128], [128, 128, 128], [128, 128, 128]],
    [[255, 255, 255], [255, 255, 255], [255, 255, 255]]
])
print(tensor)

plt.imshow(tensor, interpolation="nearest")
plt.show()
```



Knowledge refresh

Numpy

```
edades = np.array([23, 56, 67, 89, 23, 56, 27, 12, 8, 72])
filtroedades=edades > 30
filtroedades

[4] ✓ 0.0s Python
... array([False,  True,  True,  True, False,  True, False, False, False,
         True])

generos = np.array(['m', 'm', 'f', 'f', 'm', 'f', 'm', 'm', 'm', 'f'])
filtrogenero=generos == 'm'
filtrogenero

[5] ✓ 0.0s Python
... array([ True,  True, False, False,  True, False,  True,  True,  True,
        False])

Filtrocompuesto=(edades > 10) & (generos == 'f')
Filtrocompuesto

[6] ✓ 0.0s Python
... array([False, False,  True,  True, False,  True, False, False, False,
         True])
```



Knowledge refresh

Numpy

```
data = [[0, 2, 4, 6], [1, 'a', 5, 7]]
dataArray = np.array(data)
transpuesta=dataArray.T
transpuesta

[8] ✓ 0.0s Python

... array([[ '0', '1'],
         [ '2', 'a'],
         [ '4', '5'],
         [ '6', '7']], dtype='<U21')

#Cambiamos las dimensiones de una matriz.
res=dataArray.reshape(4,2)
res

[12] ✓ 0.0s Python

... array([[ '0', '2'],
         [ '4', '6'],
         [ '1', 'a'],
         [ '5', '7']], dtype='<U21')

#Convertimos un arreglo de cualquier número de dimensiones en uno de una sola dimensión
uni=dataArray.ravel()
uni

[13] ✓ 0.0s Python

... array([ '0', '2', '4', '6', '1', 'a', '5', '7'], dtype='<U21')
```



Knowledge refresh

Numpy

```
print(tensor.shape)
a,b,c =tensor.shape
print(a,b,c)
```

✓ 0.0s

(3, 3, 3)
3 3 3

```
vector2=np.zeros(3)
matriz2=np.zeros((3,3))
print(matriz2)
```

[[0. 0. 0.]
[0. 0. 0.]
[0. 0. 0.]

```
vector=np.array([2,5,10])
print(np.mean(vector))#promedio
print(np.min(vector))
print(np.max(vector))
print(np.std(vector))
```

5.666666666666667
2
10
3.2998316455372216



Knowledge refresh

Numpy-dot product

Si tenemos

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

entonces

$$\mathbf{a} \cdot \mathbf{b} = a_1b_1 + a_2b_2 + \cdots + a_nb_n$$

Es decir:

1. Multiplicar elemento a elemento.
2. Sumar todos los resultados.

El producto punto mide qué tan alineados están dos vectores.

- **grande positivo** → apuntan en dirección similar
- **cero** → son perpendiculares
- **negativo** → apuntan en direcciones opuestas

Realiza el experimento con 2 vectores del mismo y de diferente tamaño



Knowledge refresh

Numpy- Dot producto - matrices

$$\begin{matrix} & A & \\ \begin{bmatrix} 2 & 1 & 3 \\ 4 & 0 & 2 \end{bmatrix} & \times & \begin{matrix} B \\ \begin{bmatrix} 5 & 2 \\ 1 & 4 \\ 3 & 7 \end{bmatrix} \end{matrix} & = & \begin{matrix} AB \\ \begin{bmatrix} 20 & 29 \\ 26 & 22 \end{bmatrix} \end{matrix} \end{matrix}$$

$$(AB)_{11} = (2 \cdot 5) + (1 \cdot 1) + (3 \cdot 3) = 20$$



Knowledge refresh

Numpy- Dot producto

Entonces en Machine learning...

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 2 & 1 & 3 & 0 & 4 \\ 5 & 2 & 1 & 3 & 6 \\ 7 & 8 & 5 & 1 & 2 \\ 3 & 6 & 9 & 2 & 1 \\ 4 & 5 & 6 & 7 & 8 \\ 1 & 3 & 5 & 7 & 9 \\ 9 & 8 & 7 & 6 & 5 \\ 2 & 4 & 6 & 8 & 10 \end{bmatrix}$$

$$W = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix}$$

$$(10 \times 5) \cdot (5 \times 1)$$

$$XW = \begin{bmatrix} 55 \\ 130 \\ 33 \\ 53 \\ 58 \\ 54 \\ 100 \\ 95 \\ 95 \\ 110 \end{bmatrix}$$

$$(10 \times 1)$$



Knowledge refresh

Numpy

Actividad:

10 MIN

Consulte que es broadcasting en Python y mas específicamente en numpy, y haga 2 ejemplos para mostrarle al profesor



Knowledge refresh

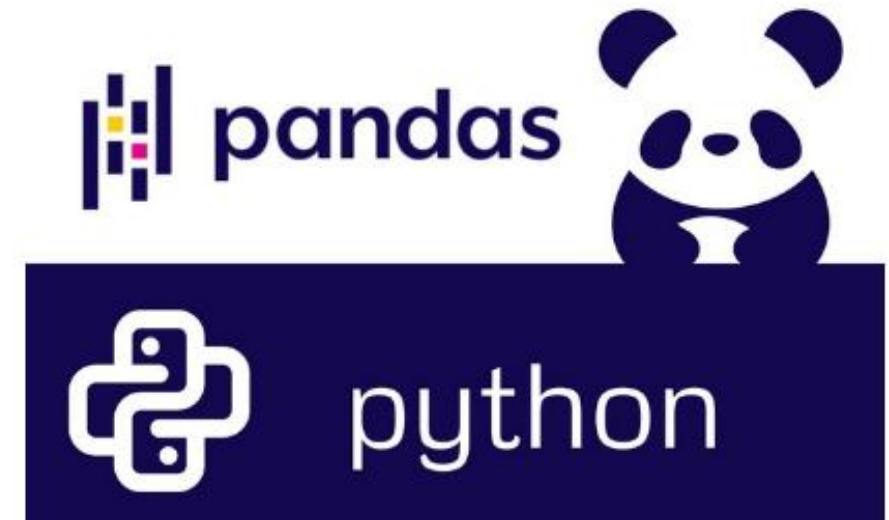
Pandas

Es una librería fundamental de Python, ampliamente utilizada para la manipulación y el análisis de datos.

- El nombre "Pandas" proviene de "Panel Data".

Pandas ofrece dos estructuras de datos clave que son el corazón de su funcionalidad:

- **Series:** Es una estructura de datos unidimensional.
- **DataFrame:** Es bidimensional, tabular y se asemeja mucho a una tabla de base de datos o una hoja de cálculo.



Knowledge refresh

Pandas

Manejo Eficiente de Datos Tabulares: La estructura DataFrame es increíblemente intuitiva y poderosa para trabajar con datos en formato de tabla. Permite organizar, limpiar y manipular grandes volúmenes de datos de manera eficiente.

- Lectura y Escritura de Datos Diversos
- Facilidad para la Limpieza y Preprocesamiento de Datos
- Integración con el Ecosistema de Python
- Comunidad Grande y Activa
- Rendimiento

	country	capital	area	population
BR	Brazil	Brasilia	8.516	200.40
RU	Russia	Moscow	17.100	143.50
IN	India	New Delhi	3.286	1252.00
CH	China	Beijing	9.597	1357.00
SA	South Africa	Pretoria	1.221	52.98
FR	Francia	París	2.534	32.80



Knowledge refresh

Pandas-Series

CREACION DE UNA SERIE

```
# Desde una lista de Python (índice por defecto)
s1 = pd.Series([10, 20, 30, 40, 50])
# Desde una lista con un índice personalizado
s2 = pd.Series([100, 200, 300], index=['a', 'b', 'c'])
# Desde un diccionario (las claves se convierten en índices)
diccionario = {'Manzana': 3, 'Banana': 5, 'Naranja': 2}
s3 = pd.Series(diccionario)
# Desde un array de NumPy
np_array = np.array([5, 10, 15, 20])
s4 = pd.Series(np_array)
s3
```

✓ 0.0s

```
Manzana    3
Banana     5
Naranja    2
dtype: int64
```

```
# Acceso por índice numérico (posición)
print("Elemento en la posición 0:", s1[0]) # Output: 3
# Acceso por etiqueta de índice
print("Valor de 'Banana':", s3['Banana']) # Output: 5
# Slicing por posición (exclusivo el final)
print("Elementos de la posición 0 a 1 (excluido 2):")
print(s3[0:2])
# Slicing por etiqueta (inclusivo el final)
print("Elementos desde 'Manzana' hasta 'Naranja' (incluido Naranja):")
print(s3['Manzana':'Naranja'])
# Indexación Booleana (filtrado)
print("Frutas con valor > 2:")
print(s3[s3 > 2])
```



Knowledge refresh

Pandas-Series

```
# Usaremos s1 de ejemplo: [10, 20, 30, 40, 50]
```

```
# Sumar un escalar a cada elemento
```

```
s_suma = s1 + 5
```

```
print("Serie + 5:\n", s_suma)
```

```
# Multiplicar Series (alineación por índice)
```

```
s5 = pd.Series([1, 2, 3, 4, 5])
```

```
s_multiplicacion = s1 * s5
```

```
print("s1 * s5:\n", s_multiplicacion)
```

```
# Operaciones estadísticas
```

```
print("Suma de s1:", s1.sum())
```

```
print("Media de s1:", s1.mean())
```

```
print("Valor máximo de s1:", s1.max())
```



Knowledge refresh

Pandas-Series

Series: Manejo de Valores Faltantes (NaN)

```
print("\n--- Manejo de Valores Faltantes ---")
s_nan = pd.Series([1, 2, np.nan, 4, np.nan, 6])
print("Serie con NaN:\n", s_nan)
```

```
# Verificar valores nulos
print("¿Hay valores nulos?:", s_nan.isnull().any())
```

```
# Eliminar valores nulos
s_sin_nan = s_nan.dropna()
print("Serie después de dropna():\n", s_sin_nan)
```

```
# Rellenar valores nulos
s_rellenada = s_nan.fillna(0)
print("Serie después de fillna(0):\n", s_rellenada)
```

Serie con NaN:

```
0    1.0
1    2.0
2    NaN
3    4.0
4    NaN
5    6.0
dtype: float64
```

¿Hay valores nulos?: True

Serie después de dropna():

```
0    1.0
1    2.0
3    4.0
5    6.0
dtype: float64
```

Serie después de fillna(0):

```
0    1.0
1    2.0
2    0.0
3    4.0
4    0.0
5    6.0
dtype: float64
```



Knowledge refresh

Pandas-DataFrame

CREACIÓN DE UN DATAFRAME

En general, existen multiples formas de crear un Dataframe, desde una lista, desde un diccionario, desde un array de numpy o incluso se pueden importar archivos externos como .csv, .xls, o sql

```
#Desde una lista
Animes_favoritos = [['Shingeki no Kyogin', 'Eren', "Shonen"], ['Naruto', 'Naruto'], ['Death note', 'Yagami',"Shonen"],["One Piece"], ["Saint Seiya", "Seiya", "Shonen", "4", "Athena"]]
df_animes = pd.DataFrame(Animes_favoritos)
df_animes
```

```
#Desde un diccionario con listas de igual longitud
dict = {"country":["Brazil", "Russia", "India", "China", "South Africa"],
        "capital":["Brasilia", "Moscow", "New Delhi", "Beijing", "Pretoria"],
        "area":[8.516, 17.10, 3.286, 9.597, 1.221],
        "population":[200.4, 143.5, 1252, 1357, 52.98]}
#Llaves→ etiquetas de las columnas
#Valores → los datos de cada columna

brics = pd.DataFrame(dict)
brics
```



Knowledge refresh

Pandas-DataFrame

#Si deseamos eliminar la primera columna de índices.
brics.index = ["BR", "RU", "IN", "CH", "SA"]

»brics

	country	capital	area	population
BR	Brazil	Brasilia	8.516	200.40
RU	Russia	Moscow	17.100	143.50
IN	India	New Delhi	3.286	1252.00
CH	China	Beijing	9.597	1357.00
SA	South Africa	Pretoria	1.221	52.98



Knowledge refresh

Pandas-DataFrame

IMPORTAR DATOS CSV

DEMO



Knowledge refresh

Pandas-DataFrame

Filtrado con condicionales:

```
import pandas as pd
import numpy as np

# Crear un DataFrame de ejemplo
data = {
    'Producto': ['Laptop', 'Mouse', 'Teclado', 'Monitor', 'Cámara', 'Auriculares',
                'Impresora', 'Disco Duro'],
    'Precio': [1200, 25, 75, 300, 150, 50, 200, 80],
    'Stock': [10, 50, 30, 20, 15, 40, 5, 45],
    'Categoria': ['Electrónica', 'Accesorios', 'Accesorios', 'Electrónica', 'Electrónica',
                 'Accesorios', 'Electrónica', 'Accesorios'],
    'Disponible': [True, True, False, True, True, False, True, True]
}
df = pd.DataFrame(data)

print("DataFrame Original:")
print(df)
```

DataFrame Original:

	Producto	Precio	Stock	Categoria	Disponible
0	Laptop	1200	10	Electrónica	True
1	Mouse	25	50	Accesorios	True
2	Teclado	75	30	Accesorios	False
3	Monitor	300	20	Electrónica	True
4	Cámara	150	15	Electrónica	True
5	Auriculares	50	40	Accesorios	False
6	Impresora	200	5	Electrónica	True
7	Disco Duro	80	45	Accesorios	True



Knowledge refresh

Pandas-DataFrame

```
...
df_precios_altos = df[condicion_precio]
# O directamente: df_precios_altos = df[df['Precio'] > 100]
print("\nDataFrame de Productos con Precio > 100:")
print(df_precios_altos)
```

DataFrame Original:

	Producto	Precio	Stock	Categoria	Disponible
0	Laptop	1200	10	Electrónica	True
1	Mouse	25	50	Accesorios	True
2	Teclado	75	30	Accesorios	False
3	Monitor	300	20	Electrónica	True
4	Cámara	150	15	Electrónica	True
5	Auriculares	50	40	Accesorios	False
6	Impresora	200	5	Electrónica	True
7	Disco Duro	80	45	Accesorios	True

--- Filtrado con una Sola Condición ---

Serie Booleana (Precio > 100):

0	True
1	False
2	False
3	True
4	True
5	False
6	True
7	False

Name: Precio, dtype: bool

DataFrame de Productos con Precio > 100:

	Producto	Precio	Stock	Categoria	Disponible
0	Laptop	1200	10	Electrónica	True
3	Monitor	300	20	Electrónica	True
4	Cámara	150	15	Electrónica	True
6	Impresora	200	5	Electrónica	True



Knowledge refresh

Pandas-DataFrame

```
...  
# Condición 2: Productos en la 'Categoría' 'Electrónica'  
df_electronica = df[df['Categoría'] == 'Electrónica']  
print("\nDataFrame de Productos de la Categoría 'Electrónica':")  
print(df_electronica)
```

DataFrame Original:

	Producto	Precio	Stock	Categoría	Disponible
0	Laptop	1200	10	Electrónica	True
1	Mouse	25	50	Accesorios	True
2	Teclado	75	30	Accesorios	False
3	Monitor	300	20	Electrónica	True
4	Cámara	150	15	Electrónica	True
5	Auriculares	50	40	Accesorios	False
6	Impresora	200	5	Electrónica	True
7	Disco Duro	80	45	Accesorios	True

DataFrame de Productos de la Categoría 'Electrónica':

	Producto	Precio	Stock	Categoría	Disponible
0	Laptop	1200	10	Electrónica	True
3	Monitor	300	20	Electrónica	True
4	Cámara	150	15	Electrónica	True
6	Impresora	200	5	Electrónica	True



Knowledge refresh

Pandas-DataFrame

FILTROS EJERCICIO



Knowledge refresh

Pandas-DataFrame

```
...  
# Condición: Productos de 'Electrónica' Y con 'Stock' mayor a 10  
df_electronica_stock_alto = df[(df['Categoria'] == 'Electrónica') & (df['Stock']  
> 10)]  
print("\nDataFrame de Productos de Electrónica con Stock > 10:")  
print(df_electronica_stock_alto)
```

DataFrame Original:

	Producto	Precio	Stock	Categoria	Disponible
0	Laptop	1200	10	Electrónica	True
1	Mouse	25	50	Accesorios	True
2	Teclado	75	30	Accesorios	False
3	Monitor	300	20	Electrónica	True
4	Cámara	150	15	Electrónica	True
5	Auriculares	50	40	Accesorios	False
6	Impresora	200	5	Electrónica	True
7	Disco Duro	80	45	Accesorios	True

DataFrame de Productos de Electrónica con Stock > 10:

	Producto	Precio	Stock	Categoria	Disponible
3	Monitor	300	20	Electrónica	True
4	Cámara	150	15	Electrónica	True

Nota: el “or” se realiza con “|”.
El negado se realiza con “~”.



Knowledge refresh

Pandas-DataFrame

Consulte como ordenar su dataframe en orden ascendente o descendente considerando una columna y aplíquelo



Knowledge refresh

Actividad

Cargar o crear un dataset para aplicar al menos 5 funcionalidades de Pandas que a ustedes les parezca interesante o no hayan probado.

Posibles fuentes de datasets:

- <https://www.kaggle.com/datasets>
- <https://ieee-dataport.org/datasets>
- <https://mlcontests.com/>



Adquisición y limpieza de datos

Con un dataset limpio y bien estructurado el performance de los modelos puede aumentar

Dataset “sucio”

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. Generar datos base
np.random.seed(41)
n_samples = 500

# Variables: Edad y Puntuación (0-100)
edad = np.random.randint(18, 70, n_samples)
puntuacion = np.random.randint(1, 100, n_samples)

# Crear el target (Compra: 1 si es joven y gasta mucho, o si es mayor y gasta poco)
# Añadimos ruido para que el accuracy no sea perfecto
target = ((edad < 40) & (puntuacion > 50) | (edad > 40) & (puntuacion < 30)).astype(int)
ruido = np.random.choice([0, 1], size=n_samples, p=[0.85, 0.15])
target = np.logical_xor(target, ruido).astype(int)

df = pd.DataFrame({'Edad': edad, 'Puntuacion': puntuacion, 'Compro': target})

# --- ENSUCIAR EL DATASET ---

# A. Insertar valores nulos (NaN)
df.loc[df.sample(frac=0.1).index, 'Edad'] = np.nan

# B. Insertar Outliers (Valores imposibles)
df.loc[0:5, 'Edad'] = 250 # Alguien de 250 años
df.loc[6:10, 'Puntuacion'] = -50 # Puntuación negativa

# C. Insertar Duplicados
df = pd.concat([df, df.iloc[:20]], ignore_index=True)

# D. Columna categórica con errores de escritura
países = ['España', 'Mexico', 'Colombia', 'Argentina']
df['Pais'] = np.random.choice(países, len(df))
df.loc[df.sample(frac=0.05).index, 'Pais'] = 'Españaa' # Error tipográfico

print("Dataset sucio creado. Primeras filas:")
print(df.head())
```

Resultado algoritmo ML

```
#Algoritmo de ML
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# 1. Limpieza básica para que el modelo pueda ejecutarse
# Rellenamos los nulos de 'Edad' con la mediana (para evitar que el outlier de 250 afecte tanto)
df_limpio = df.copy()
df_limpio['Edad'] = df_limpio['Edad'].fillna(df_limpio['Edad'].median())

# Convertimos la columna 'Pais' a números (Label Encoding rápido)
df_limpio['Pais'] = df_limpio['Pais'].astype('category').cat.codes

# 2. División de datos (Features X y Target y)
X = df_limpio.drop('Compro', axis=1)
y = df_limpio['Compro']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Entrenamiento del Modelo
modelo = RandomForestClassifier(n_estimators=100, random_state=42)
modelo.fit(X_train, y_train)

# 4. Predicción y Evaluación
y_pred = modelo.predict(X_test)

print(f"Accuracy final: {accuracy_score(y_test, y_pred):.2f}")
print("\nMatriz de Confusión:")
print(confusion_matrix(y_test, y_pred))
```

Accuracy final: 0.78



Adquisición y limpieza de datos

Dataset limpio

```
# 1. Separar las clases
df_mayoritaria = df_final[df_final.Compro == 0]
df_minoritaria = df_final[df_final.Compro == 1]

# 2. Sobremuestrear la clase minoritaria
df_minoritaria_upsampled = resample(df_minoritaria,
                                    replace=True,      # duplicar muestras
                                    n_samples=len(df_mayoritaria), # igualar a la mayoritaria
                                    random_state=42)

# 3. Combinar de nuevo
df_balanceado = pd.concat([df_mayoritaria, df_minoritaria_upsampled])

# 4. Re-entrenar
X_bal = df_balanceado.drop(['Compro', 'Pais'], axis=1)
y_bal = df_balanceado['Compro']
```

Resultado algoritmo ML

```
X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(X_bal, y_bal, test_size=0.2, random_state=42)

modelo_final = RandomForestClassifier(n_estimators=100, random_state=42)
modelo_final.fit(X_train_b, y_train_b)

# 5. Evaluación final
y_pred_final = modelo_final.predict(X_test_b)
print(f"Accuracy con datos limpios y balanceados: {accuracy_score(y_test_b, y_pred_final):.2f}")
print("\nReporte de Clasificación:")
print(classification_report(y_test_b, y_pred_final))
```

Accuracy con datos limpios y balanceados: 0.82



Adquisición y limpieza de datos

TIPOS DE DATOS

Estructurados:

Poseen formato predefinido y una organización clara y consistente.

Altamente organizados, lo que permite una fácil manipulación y búsqueda.

Menos flexible: Su estructura predefinida puede limitar su uso para propósitos distintos a los originales.

Definición previa: se debe definir la estructura de los datos (columnas, tipos de datos, relaciones) antes de poder almacenar cualquier información.

Dificultad de cambio: Alterar este esquema (añadir una nueva columna, cambiar un tipo de dato) después de que los datos ya están almacenados puede ser una operación costosa y compleja, especialmente en grandes volúmenes de datos.



Adquisición y limpieza de datos

TIPOS DE DATOS

Estructurados:

Bases de datos relacionales (SQL): MySQL, PostgreSQL, Oracle, SQL Server. Contienen información como registros de clientes, transacciones financieras, inventarios, etc.

Hojas de cálculo: Archivos CSV, Archivos de Excel o Google Sheets con datos organizados en celdas



Adquisición y limpieza de datos

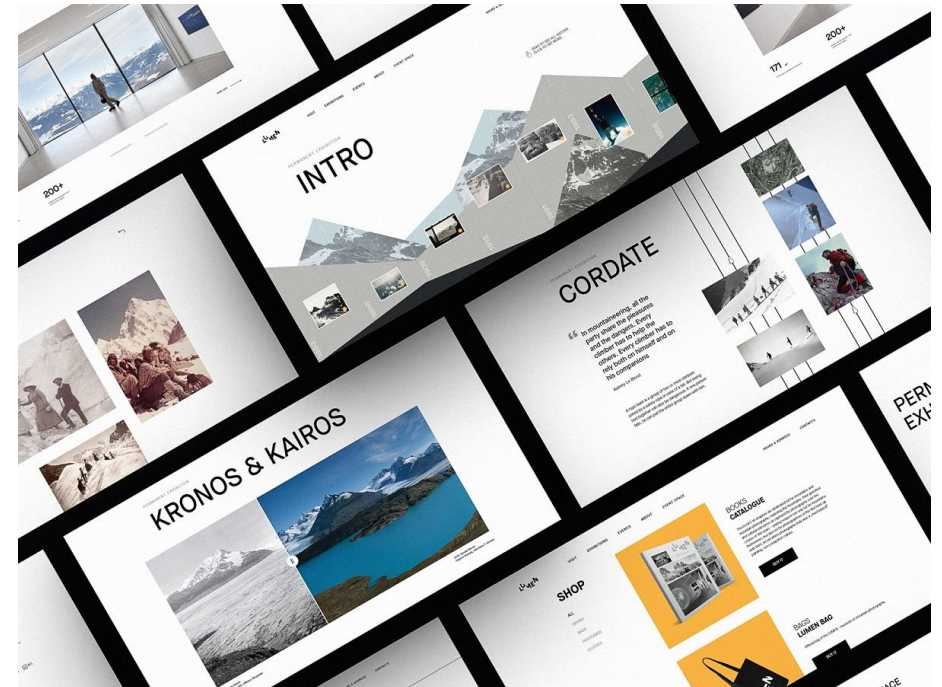
TIPOS DE DATOS

Estructurados:

Fuentes de datos estructurados:

Formularios web: Resultados de encuestas o registros en línea con campos definidos.

Sensores y dispositivos IoT: Datos de geolocalización, temperatura, presión, etc., cuando se presentan en un formato tabular.



Adquisición y limpieza de datos

TIPOS DE DATOS

Semiestructurados:

Tienen cierta estructura, pero no siguen un esquema fijo y rígido como los datos estructurados.

Utilizan etiquetas o marcadores para organizar y separar elementos de datos, pero no necesariamente se organizan en tablas.

- Archivos XML (Extensible Markup Language): Utilizan etiquetas para definir y organizar los datos.
- Archivos JSON (JavaScript Object Notation): Formato ligero para el intercambio de datos, muy común en APIs web.



Adquisición y limpieza de datos

TIPOS DE DATOS

Semiestructurados:

Tienen cierta estructura, pero no siguen un esquema fijo y rígido como los datos estructurados.

Utilizan etiquetas o marcadores para organizar y separar elementos de datos, pero no necesariamente se organizan en tablas.

- Archivos XML (Extensible Markup Language): Utilizan etiquetas para definir y organizar los datos.
- Archivos JSON (JavaScript Object Notation): Formato ligero para el intercambio de datos, muy común en APIs web.



Adquisición y limpieza de datos

TIPOS DE DATOS

No estructurados:

No poseen formato predefinido ni una estructura organizacional clara.

No se ajustan a un modelo de datos tradicional y pueden ser difíciles de categorizar y analizar.

Son abundantes y se generan de forma continua (se estima que representan el 80-90% de los datos).

Flexibles y escalables: Al no tener una estructura rígida, son más adaptables a nuevas necesidades de análisis.



Adquisición y limpieza de datos

TIPOS DE DATOS

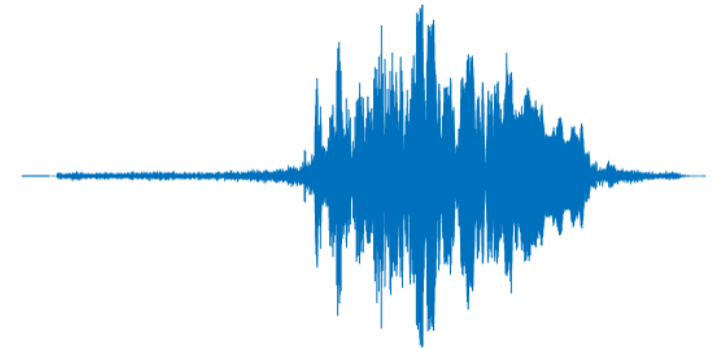
No estructurados:

Texto: Documentos (PDF, Word), correos electrónicos, publicaciones en redes sociales, comentarios, reseñas, artículos, libros.

Multimedia: Imágenes (JPG, PNG), videos (MP4, AVI), audio (MP3, WAV).

Datos de comunicación: Mensajes de chat, grabaciones de llamadas.

Datos científicos: Informes sísmicos, datos de exploración espacial, resonancias magnéticas, radiografías.



Gracias...

